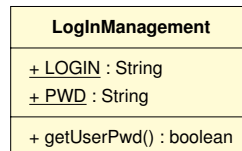


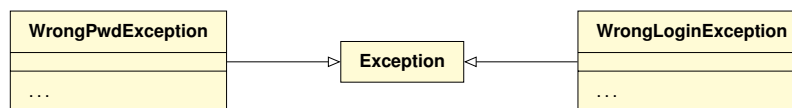
Exercice 1 : Saisie et contrôle d'un mot de passe

Q1. Écrivez une classe `LogInManagement` qui demande en boucle un nom d'utilisateur (`login`) et un mot de passe (`pwd`) jusqu'à recevoir un couple correct. Un seul couple sera référencé comme correct et défini comme constante de la classe. La saisie au clavier doit être réalisée via un flux en entrée associé à `System.in`.



Noter que la méthode `getUserPwd()` ne doit pas gérer les exceptions (pas d'instructions `try/catch`), mais seulement une propagation à la méthode principale qui traitera les exceptions.

Q2. Définissez les exceptions `WrongLoginException` et `WrongPwdException` qui sont levées lorsque l'utilisateur saisit un login inexistant ou un mot-de-passe incorrect. Toute exception levée doit afficher un message spécifique.



Q3. Modifiez votre classe `LogInManagement` de telle sorte que la méthode `getUserPwd()` ait un type de retour `void` et lance explicitement une exception de type `WrongLoginException` ou de type `WrongPwdException` le cas échéant. Les captures et les traitements des exceptions surviennent dans la méthode principale.

Q4. De la même manière, ajoutez l'exception `WrongInputLengthException` qui survient lorsque le nom d'utilisateur ou le mot de passe dépasse 10 caractères. Modifiez votre classe `LogInManagement` en conséquence pour prendre en compte ce nouveau type d'exception.

Q5. Réalisez un `commit` associé au message : `"tp00-09::exo-login"`. Soumettez ensuite l'ensemble des `commit` au dépôt.

Exercice 2 : Extraction d'information

Le fichier `DicoJava.txt` contient une suite de lignes de texte, chaque ligne étant composée de 3 champs séparés par une tabulation. Les champs sont dans l'ordre un mot réservé du langage, son type et une description. Voici un exemple du début de fichier :

```

abstract Mot-clé Déclaration d'une méthode ou d'une classe abstraite.
assert Mot-clé Assertion
break Mot-clé Interrompt une boucle ou un choix multiple.

```

Q1. Écrivez une classe `DicoJava` dont la méthode principale va lire ligne à ligne ce fichier, extraire les mots réservés qui sont des mots-clés (leur type, seconde colonne), en constituer une liste ordonnée, puis sauvegarder cette liste dans un fichier texte `MotsJava.txt`, à raison d'un mot par ligne.

Q2. Réalisez un `commit` associé au message : `"tp00-09::exo-dico"`. Soumettez ensuite l'ensemble des `commit` au dépôt.

Exercice 3 : Manipulation du système de fichiers

Le but de cet exercice est d'explorer et filtrer une sous-partie d'un système de fichiers afin d'afficher les éléments satisfaisants certaines conditions. Pour ce faire, l'utilisation de la classe `File` du paquetage `java.io` est incontournable. Voici un extrait des méthodes qui pourraient vous être utiles :

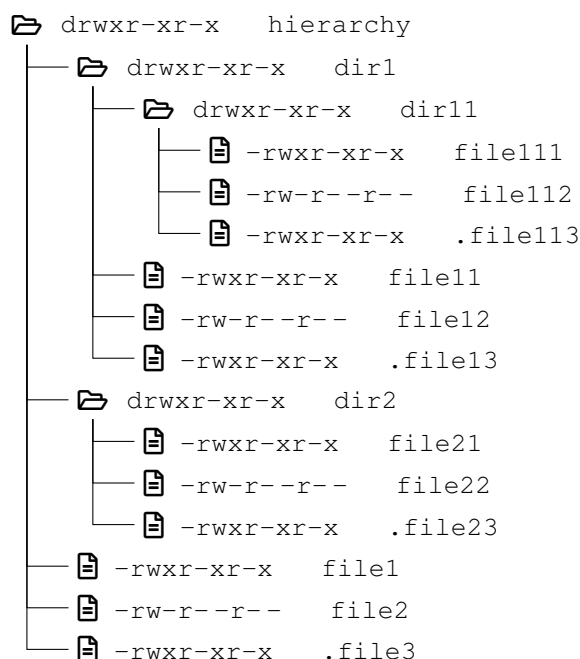
Class File

	<code>File(String pathname)</code> Creates a new File instance by converting the given pathname string into an abstract pathname.
boolean	<code>canExecute()</code> Tests whether the application can execute the file denoted by this abstract pathname.
boolean	<code>exists()</code> Tests whether the file or directory denoted by this abstract pathname exists.
String	<code>getName()</code> Returns the name of the file or directory denoted by this abstract pathname.
boolean	<code>isDirectory()</code> Tests whether the file denoted by this abstract pathname is a directory.
boolean	<code>isFile()</code> Tests whether the file denoted by this abstract pathname is a normal file.
boolean	<code>isHidden()</code> Tests whether the file named by this abstract pathname is a hidden file.
String[]	<code>list()</code> Returns an array of strings naming the files and directories in the directory denoted by this abstract pathname.
File[]	<code>listFiles()</code> Returns an array of abstract path names denoting the files in the directory denoted by this abstract pathname.

Q1. Au sein d'une classe `ListExecutable`, écrivez une méthode `printExecutable(...)` qui, à partir d'un chemin donné, permet de lister le chemin absolu de tous les fichiers que l'on peut exécuter à condition qu'ils ne soient pas cachés, quel que soit le niveau de profondeur par rapport au chemin donné.

```
void printExecutable(File file)
```

Par exemple, l'arborescence suivante est composée de différents niveaux de profondeur, et chaque répertoire contient un fichier exécutable, un fichier non exécutable et un fichier caché exécutable (élément dont le nom commence par le caractère "."). La méthode mène à l'affichage de 4 éléments, à savoir : `file111`, `file11`, `file21` et `file1`.



Une classe `HierarchyCreation` est fournie sous forme d'une archive `jar`. Cette classe bénéficie d'une méthode

de classe `hierarchyCreation(...)` qui génère l'arborescence décrite précédemment à un chemin fournit en paramètre.

Class `HierarchyCreation`

<code>void</code>	<code>hierarchyCreation(String aPath)</code> Generates a specific file hierarchy at the given location.
-------------------	--

Q2. Réalisez un `commit` associé au message : `"tp00-09::exo-arborescence"`. Soumettez ensuite l'ensemble des `commit` au dépôt.

Exercice 4 : Fichier csv et classe Scanner

Le fichier `testScan.csv` est un fichier CSV qui est organisé de la manière suivante :

- toutes les lignes commencent par un chiffre représentant le type de données : 1 pour les booléens, 2 pour les entiers, sinon les données seront considérées comme des chaînes de caractères ;
- le second champ de chaque ligne est un entier représentant le nombre de données sur la ligne courante ;
- tous les autres champs constituent une liste des données du type décrit dans le premier champ et en nombre mentionné dans le second champ.

Q1. Écrivez une classe `Validation` munie d'une unique méthode permettant de vérifier la validité d'un fichier dont le nom est passé en paramètre :

```
static boolean checkFile(String)
```

Cette méthode renvoie `true` si la structure est correcte, sinon elle lève une exception (elle ne renvoie donc jamais `false`). Les exceptions susceptibles d'être levées sont :

- `InputMismatchException` si le type de donnée qu'on essaie de récupérer n'est pas le bon ;
- `InvalidStructureException` si la structure n'est pas correcte/conforme (à définir vous-même) ;
- `FileNotFoundException` si le fichier indiqué est introuvable.

Toutes les captures et traitements des exceptions doivent être effectués dans le programme principal.

Q2. Réalisez un `commit` associé au message : `"tp00-08::exo-scanner"`. Soumettez ensuite tous les `commit` au dépôt.